

Introduction to Swings

Origin of Swings

In the early versions of Java, graphical user interfaces (GUIs) were created using the **Abstract Window Toolkit (AWT)**. AWT provided basic tools like buttons, windows, and dialog boxes. However, AWT had **several limitations** because it relied on **platform-specific "peers"** — that is, when you created a button in Java, AWT would actually create a native button from the operating system (like Windows or Linux).

A **peer** is just the *real button provided by your OS*.

- You write: `Button b = new Button("Click");`
- AWT asks the OS → “Give me a button”
- OS creates it → That is the **peer**

Problems with AWT:

1. Inconsistency Across Platforms:

- Since components were based on the local OS, they could look or behave differently on different systems.
- This broke Java's philosophy: **"Write once, run anywhere."**

2. Fixed Look and Feel:

- The appearance of AWT components couldn't be changed easily because they were controlled by the operating system.

3. Heavyweight Components:

- AWT components are called **heavyweight** because they use native (non-Java) resources.
- This led to layout issues and restrictions — for example, heavyweight components were always rectangular and opaque, which limited design flexibility.

In AWT, components like buttons, text fields, windows, etc. are called **heavyweight** because: They are created and handled by the **operating system (OS)**, not fully by Java.

So when you add a button:

- Java says: “OS, please create this button”
- The OS creates a *real* button using its own system resources

To overcome these issues, **Swing** was introduced as a new GUI toolkit for Java:

- Swing components are **lightweight** (written entirely in Java).
- They do **not rely on native peers**, so they look and behave **consistently** across all platforms.
- Swing supports **pluggable look and feel**, allowing developers to change the appearance of components easily.

Swing Packages

Swing is a **GUI (Graphical User Interface)** toolkit in Java used to create windows, buttons, menus, and more — all in **Java code** (without depending on the operating system like AWT does).

Main Package: javax.swing - This is the **core package** of Swing. It contains classes to create:

- JFrame – a main window
 - JButton – a button
 - JLabel – a label
 - JCheckBox – a checkbox
 - JTextField – a text field
- ...and many more components.

Other Swing-Related Packages: These are **supporting packages** that add extra features to Swing components:

Package Name	Purpose
javax.swing.border	To create borders for components (like beveled or titled borders).
javax.swing.colorchooser	For color selection dialogs (JColorChooser).
javax.swing.event	Handles events (like button clicks or list selections).
javax.swing.filechooser	For file open/save dialogs (JFileChooser).
javax.swing.plaf.*	PLAF = "Pluggable Look and Feel" – change how Swing components look.
javax.swing.table	For creating and managing tables (JTable).
javax.swing.text	Handles styled or editable text components.
javax.swing.tree	For tree structures (JTree).
javax.swing.undo	Supports undo and redo functionality.

Event Handling

Event handling means **responding to user actions** like clicking a button, typing in a text field, or closing a window.

In Swing (and AWT), Java uses the **Delegation Event Model**:

- When the user interacts with a GUI component (like a button), an **event object** is created (e.g., `ActionEvent`).
- This event is **sent to an event listener** (e.g., `ActionListener`), which handles it.

```
import java.awt.*;           // For layout managers like FlowLayout
import java.awt.event.*;     // For(ActionEvent and ActionListener
import javax.swing.*;        // For Swing components like JButton, JLabel
class EventDemo implements ActionListener {
    JLabel jlab; // Label to show which button was pressed
    EventDemo() {
        JFrame jfrm = new JFrame("An Event Example");
        jfrm.setLayout(new FlowLayout()); // Layout type-left to right
        jfrm.setSize(220, 90);           // Set window size
        jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Exit on close
```

```
JButton jbtnAlpha = new JButton("Alpha");
```

```
JButton jbtnBeta = new JButton("Beta");
```

```
jbtnAlpha.addActionListener(this) ; //Registers the current object to handle button clicks.
```

```
jbtnBeta.addActionListener(this) ; //connects the button to an event handler.
```

```
jfrm.add(jbtnAlpha);    // Add buttons to window
```

```
jfrm.add(jbtnBeta);
```

```
jlab = new JLabel("Press a button."); // Initial label
```

```
jfrm.add(jlab);
```

```
jfrm.setVisible(true);    // Show the window
```

```
} }
```

actionPerformed() is the **event handler** method for button click events when using **ActionListener**.

```
public void actionPerformed(ActionEvent ae) {  
    String action = ae.getActionCommand(); // Gets the text of the button clicked (Alpha  
                                           or Beta).
```

```
    if (action.equals("Alpha"))  
        jlab.setText("Alpha was pressed.");  
    else if (action.equals("Beta"))  
        jlab.setText("Beta was pressed.");  
    }  
}
```

Think of a **doorbell**:

- Button = doorbell 
- `addActionListener()` = connecting the bell to the house wiring
- `actionPerformed()` = sound that plays when pressed

Class Demo

```
{  
    public static void main(String args[]) {  
        EventDemo e= new EventDemo(); // Create and show GUI  
    }  
}
```



An Event Example



Alpha

Beta

Alpha was pressed.



An Event Example



Alpha

Beta

Beta was pressed.

Concept

`JButton`

`ActionListener`

`actionPerformed()`

`jlab.setText()`

`SwingUtilities.invokeLater()`

Meaning

A clickable button

Interface that responds to button click events

The method that runs when a button is clicked

Changes the label message

Makes sure GUI is created on the correct thread

Exploring Swing

JLabel

- JLabel is a Swing component used to **display text, images, or both**.
- It does **not react to user input** — it's a *passive* component.

Common Constructors of JLabel:

Constructor	Description
JLabel(String text)	Displays text only
JLabel(Icon icon)	Displays image only
JLabel(String text, Icon icon, int alignment)	Displays text with an image and aligns it

alignment can be: JLabel.LEFT, CENTER, RIGHT, etc.

These constants are defined in SwingConstants.

ImageIcon

- ImageIcon is a class that implements the Icon interface.
- It is used to load an image from a file or URL.
- Constructor:

```
ImageIcon icon = new ImageIcon("image.jpg");
```

```
ImageIcon ii = new ImageIcon("france.gif"); // Load image
```

```
JLabel jl = new JLabel("France", ii, JLabel.CENTER); // Create  
label with image and text
```

```
add(jl); // Add to frame
```

```
import java.awt.*;
import javax.swing.*;

public class Demo {
    public static void main(String[] args) {
        makeGUI();
    }

    private static void makeGUI() {
        // Create the frame

        JFrame frame = new JFrame("JLabel Demo");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(300, 200);
        frame.setLayout(new FlowLayout());
    }
}
```

// Create an icon

```
ImageIcon ii = new ImageIcon("C:\\Users\\Admin\\OneDrive\\Desktop\\1.jpg");
```

// Create a label with text, icon, and alignment

```
JLabel jl = new JLabel("France", ii, JLabel.CENTER);
```

// Add the label to the frame

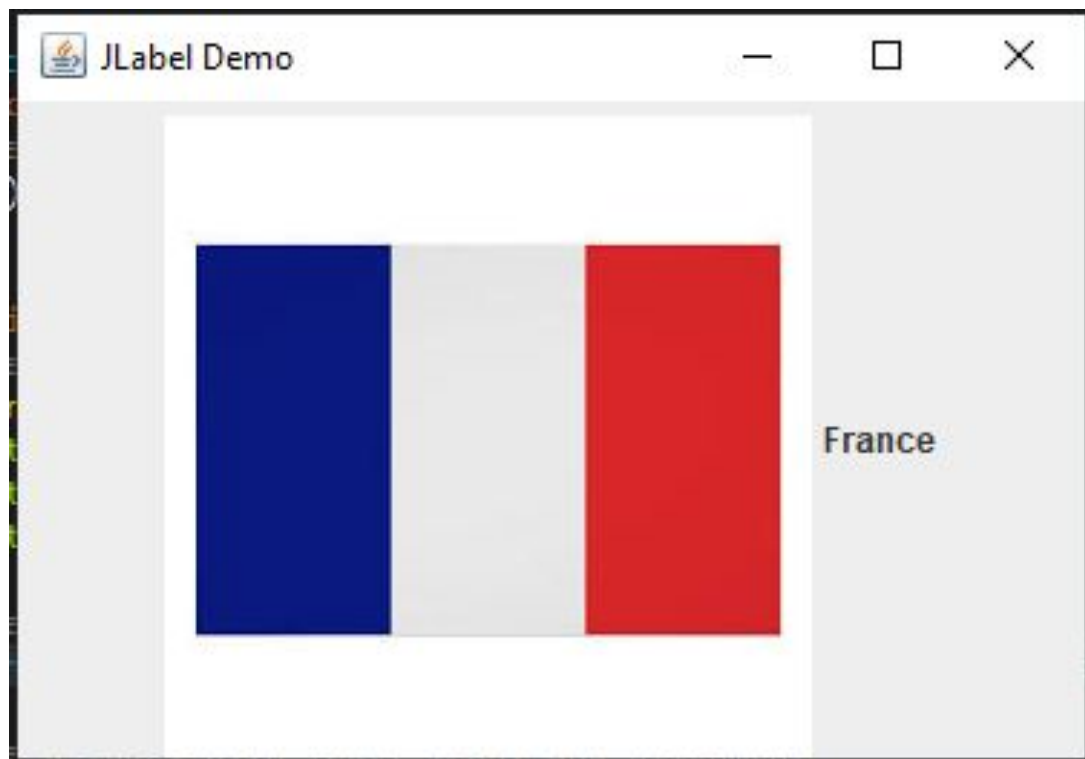
```
frame.add(jl);
```

// Make the frame visible

```
frame.setVisible(true);
```

```
}
```

```
}
```



JTextField

JTextField is a Swing component that lets the user **enter a single line of text** in a graphical user interface (GUI).

It is one of the **most commonly used input components** in Java Swing applications.

- JTextField is a subclass of JTextComponent.
- JTextComponent provides **basic text handling features**, like selecting, editing, and copying text.
- Internally, JTextField uses the **Document interface** to manage the content (text data) inside it.

Constructors of JTextField

- `JTextField(int cols)` //Creates an **empty** text field that is wide enough for **cols** **characters**.
- `JTextField(String str)` //Creates a text field with **initial text str** and sizes it to fit that text.
- `JTextField(String str, int cols)` //Creates a text field with **text str** and width of **cols** **characters**.

Examples:

```
JTextField tf1 = new JTextField(10);           // Empty, width: 10 columns
```

```
JTextField tf2 = new JTextField("Hello");       // Pre-filled with "Hello"
```

```
JTextField tf3 = new JTextField("Hi", 15);      // Pre-filled with "Hi", width: 15
```

Event Handling in JTextField

1. ActionEvent

- Triggered when the user **presses ENTER** in the text field.
- You can handle this with an **ActionListener**.

Example:

```
JTextField jtf = new JTextField(15);  
  
jtf.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent ae) {  
        System.out.println("You typed: " + jtf.getText());  
    }  
});
```

```
import java.awt.*;

import java.awt.event.*;

import javax.swing.*;

public class Demo implements ActionListener {

    JTextField jtf;

    JFrame frame;

    public Demo() {

        frame = new JFrame("JTextField Demo");

        frame.setLayout(new FlowLayout());

        frame.setSize(300, 100);

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
jtf = new JTextField(15);           // Create and add the text field  
frame.add(jtf);
```

```
jtf.addActionListener(this);       // Add ActionListener  
frame.setVisible(true);  
}
```

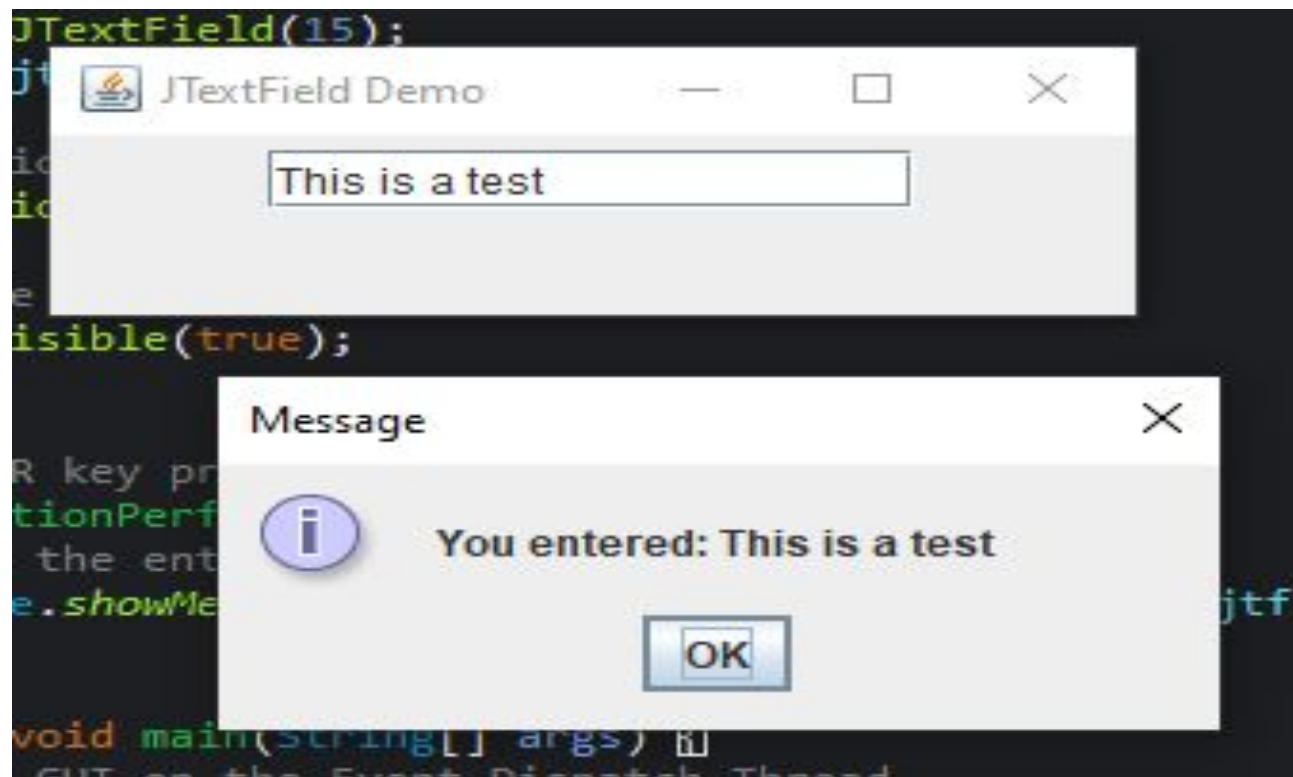
```
public void actionPerformed(ActionEvent ae) {  
    // Display the entered text in a dialog  
    JOptionPane.showMessageDialog(frame, "You entered: " + jtf.getText());  
}
```

```
public static void main(String[] args) {
```

```
    Demo d= new Demo();
```

```
}
```

```
}
```



The Swing Buttons

Swing defines four types of buttons: **JButton**, **JToggleButton**, **JCheckBox**, and **JRadioButton**.

All are subclasses of the **AbstractButton** class, which extends **JComponent**. Thus, all buttons share a set of common traits.

AbstractButton contains many methods that allow you to control the behavior of buttons.

For example, you can define different icons that are displayed for the button when it is disabled, pressed, or selected. Another icon can be used as a rollover icon, which is displayed when the mouse is positioned over a button. The following methods set these icons:

void setDisabledIcon(Icon di)

void setPressedIcon(Icon pi)

void setSelectedIcon(Icon si)

void setRolloverIcon(Icon ri)

Here, di, pi, si, and ri are the icons to be used for the indicated purpose.

The text associated with a button can be read and written via the following methods:

String getText()

void setText(String str)

Here, str is the text to be associated with the button.

JButton

The JButton class in **Swing** is used to create **push buttons** in Java GUI applications. It can display:

- **Text**
- **Icon**
- **Both text and icon**

When a button is clicked, it **generates an ActionEvent**, which you can capture and handle using an **ActionListener**.

JButton Constructors:

JButton(Icon icon)

JButton(String str)

JButton(String str, Icon icon)

Here, str and icon are the string and icon used for the button.

```
import javax.swing.*;

class ButtonIconExample {

    public static void main(String[] args) {

        JFrame frame = new JFrame("Icon Button");

        ImageIcon icon = new ImageIcon("image.png"); // path to image

        JButton btn = new JButton(icon);

        frame.add(btn);

        frame.setSize(200, 200);

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.setVisible(true);

    }

}
```

```
import javax.swing.*;

class ButtonTextIconExample {

    public static void main(String[] args) {

        JFrame frame = new JFrame("Text + Icon Button");

        ImageIcon icon = new ImageIcon("image.png");

        JButton btn = new JButton("Save", icon);

        frame.add(btn);

        frame.setSize(250, 200);

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.setVisible(true);

    }

}
```

Key Methods

- `setActionCommand(String command)` – Sets a unique command string for the button (optional).
- `getActionCommand()` – Gets the command string when the button is clicked.
- `addActionListener(ActionListener l)` – Registers an event handler for the button.

JToggleButton

- A JToggleButton looks like a normal button.
- But unlike JButton, it **stays pressed** once clicked — like a light switch.
- Clicking it again **releases it**, toggling its state between **on** and **off**.
- It fires both:
 - a. ActionEvent — like a normal button.
 - b. ItemEvent — useful to check if the button is **selected (pressed)** or not.

isSelected()- Returns true if the button is pressed

addItemListener()- Used to listen to selection/deselection

getItem()- Gets the button from the event object

It's a button with **two states**:

- ON (selected)
- OFF (not selected)

Like a **switch**

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;

class ToggleDemo implements ActionListener {

    JToggleButton toggle;

    JLabel label;

    ToggleDemo() {

        JFrame frame = new JFrame("Toggle Button Example");

        frame.setLayout(new FlowLayout());

        frame.setSize(300, 150);

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        toggle = new JToggleButton("OFF"); // initial text

        label = new JLabel("State: OFF");
```

```
toggle.addActionListener(this);  
  
frame.add(toggle);  
  
frame.add(label);  
  
frame.setVisible(true);  
  
}  
  
public void actionPerformed(ActionEvent ae) {  
  
    if (toggle.isSelected()) {  
  
        toggle.setText("ON");  
  
        label.setText("State: ON");  
  
    }  
  
}
```

```
else {  
    toggle.setText("OFF");  
    label.setText("State: OFF");  
}  
}  
  
public static void main(String[] args) {  
    new ToggleDemo();  
}  
}
```

JCheckBox

- A **check box** lets the user **select or deselect** an option.
- It can be **on** (checked) or **off** (unchecked), just like a toggle button.
- Internally, JCheckBox extends JToggleButton, so it supports two-state behavior.
- **When toggled**, it generates an ItemEvent.

`isSelected()`- Returns true if the button is pressed

`addItemListener()`- Used to listen to selection/deselection

`getText()`- Returns the label of the checkbox

```
import javax.swing.*;

import java.awt.FlowLayout;

import java.awt.event.*;

public class Test implements ItemListener {

    JCheckBox cb1, cb2;

    JLabel label;

    public Test() {

        JFrame frame = new JFrame("Checkbox Demo");

        cb1 = new JCheckBox("Java");

        cb2 = new JCheckBox("Python");
```

```
label = new JLabel("Selected: ");
    cb1.addItemListener(this);
    cb2.addItemListener(this);

    frame.add(cb1);
    frame.add(cb2);
    frame.add(label);

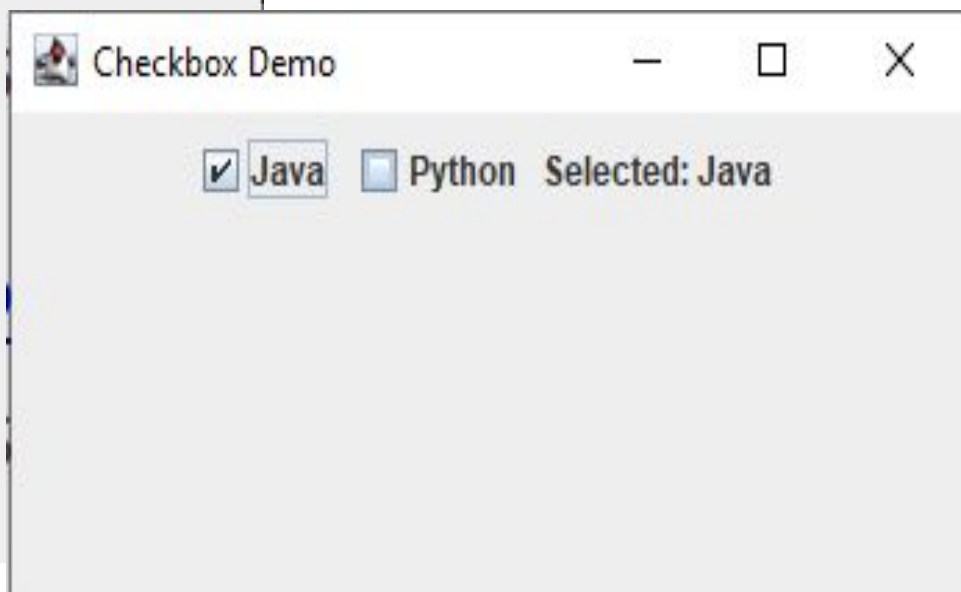
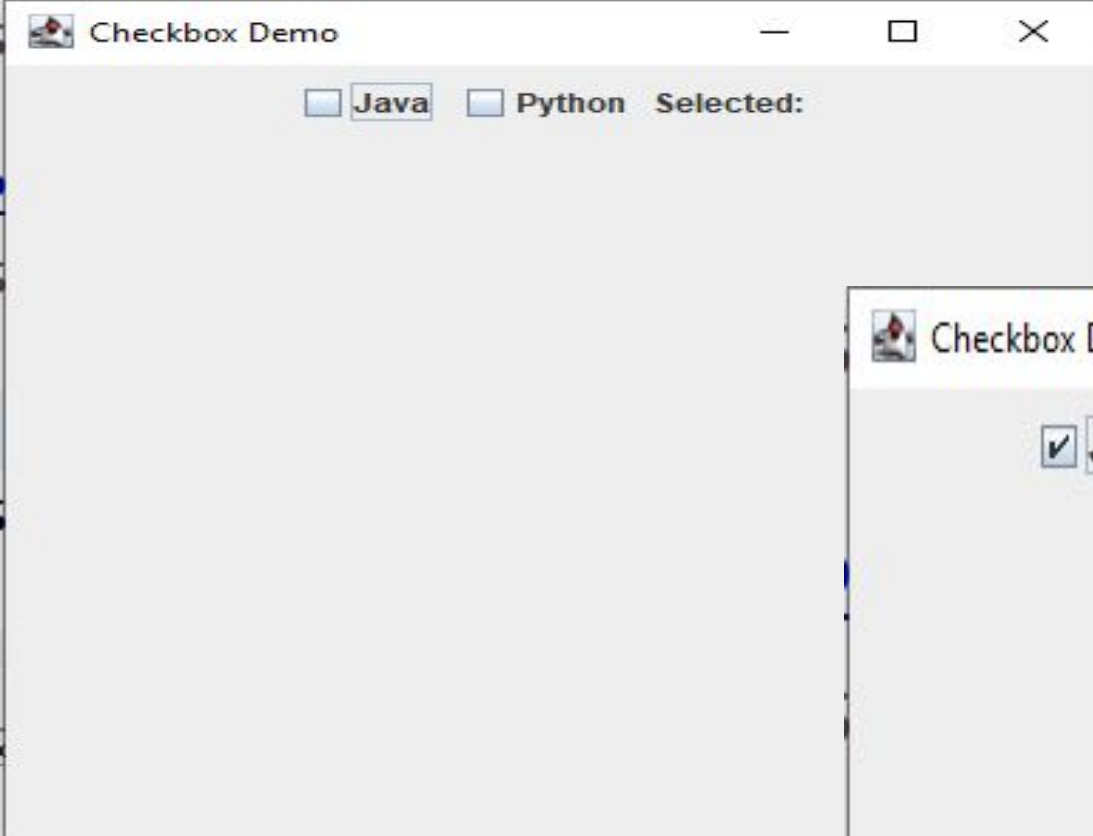
    frame.setSize(400, 400);
    frame.setLayout(new FlowLayout());
    frame.setVisible(true);
}
public void itemStateChanged(ItemEvent e) {
    String result = "Selected: ";

    if (cb1.isSelected()) {
        result += cb1.getText() + " ";
    }
}
```

```
if (cb2.isSelected()) {  
    result += cb2.getText() + " ";  
}
```

```
    label.setText(result);  
}
```

```
public static void main(String[] args) {  
    new Test(); // create object  
}  
}
```



Radio Buttons

Radio buttons let a user **choose only one option** from a group. For example, if you're selecting a gender:

- ☐ Male
- ☐ Female
- ☐ Other

You can choose **only one**. If you select another, the previous one is **automatically unselected**.

In Swing, this is done using:

- JRadioButton for each button
- ButtonGroup to group them

```
import javax.swing.*;
import java.awt.FlowLayout;
import java.awt.event.*;

public class RadioDemo implements ActionListener {
    JRadioButton r1, r2, r3;
    JLabel label;

    public RadioDemo() {
        JFrame frame = new JFrame("Radio Button Demo");
        r1 = new JRadioButton("Java");
        r2 = new JRadioButton("Python");
        r3 = new JRadioButton("C++");
```

```
// Group them (only one can be selected)
```

```
ButtonGroup bg = new ButtonGroup();
```

```
bg.add(r1);
```

```
bg.add(r2);
```

```
bg.add(r3);
```

```
label = new JLabel("Select a language");
```

```
r1.addActionListener(this);
```

```
r2.addActionListener(this);
```

```
r3.addActionListener(this);
```

```
frame.add(r1);
```

```
frame.add(r2);
```

```
frame.add(r3);
```

```
frame.add(label);
```

```
frame.setSize(400, 400);
    frame.setLayout(new FlowLayout());
    frame.setVisible(true);
}
public void actionPerformed(ActionEvent e) {
    if (r1.isSelected()) {
        label.setText("Selected: " + r1.getText());
    } else if (r2.isSelected()) {
        label.setText("Selected: " + r2.getText());
    } else if (r3.isSelected()) {
        label.setText("Selected: " + r3.getText());
    }
}
public static void main(String[] args) {
    new RadioDemo();
}
}
```



Radio Button Demo



Java



Python



C++

Selected: C++

JTabbedPane

JTabbedPane is a **container that lets you organize components into different tabs**. Only one tab's content is visible at a time, like you see in web browsers or settings dialogs.

Create the tabbed pane:

```
JTabbedPane jtp = new JTabbedPane();
```

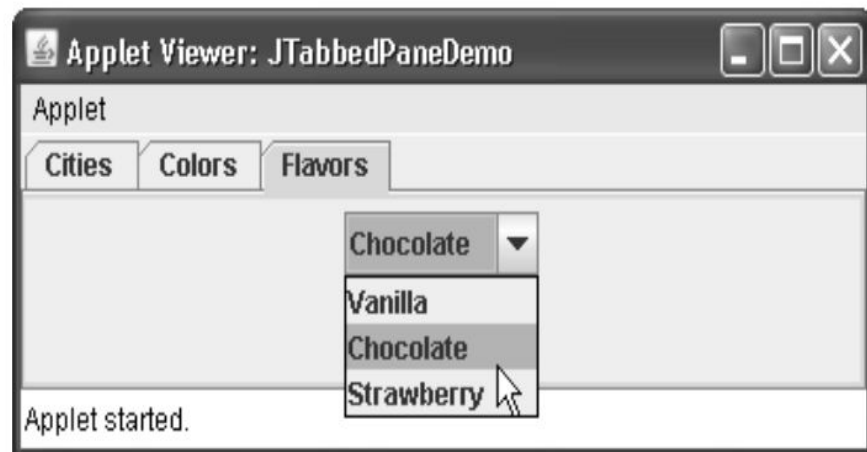
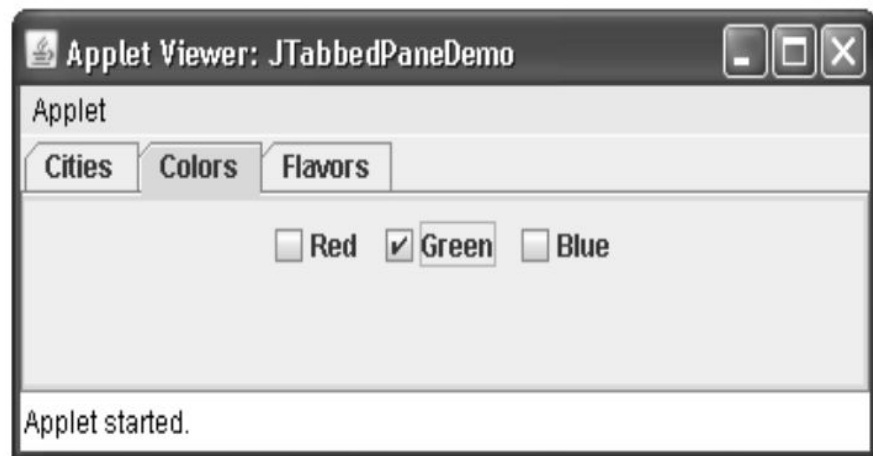
Add tabs using:

```
addTab(String title, Component comp);
```

Attach the tabbed pane to your main window or applet.

A window with **3 tabs**:

- **Cities** → Shows 4 buttons (New York, London, Hong Kong, Tokyo)
- **Colors** → Shows 3 checkboxes (Red, Green, Blue)
- **Flavors** → Shows 1 dropdown with Vanilla, Chocolate, Strawberry



```
import javax.swing.*;

import java.awt.event.*;

public class JTabbedPaneDemo implements ActionListener {

    JFrame frame;

    JTabbedPane tabbedPane;

    public JTabbedPaneDemo() {

        frame = new JFrame("TabbedPane Example");

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.setSize(400, 200);
```

```
tabbedPane = new JTabbedPane();

tabbedPane.addTab("Cities", new CitiesPanel());

tabbedPane.addTab("Colors", new ColorsPanel());

tabbedPane.addTab("Flavors", new FlavorsPanel());

frame.add(tabbedPane);

frame.setLocationRelativeTo(null); // Center the window

frame.setVisible(true);

}

public void actionPerformed(ActionEvent e) {

// No actions are added in this simple demo

}
```

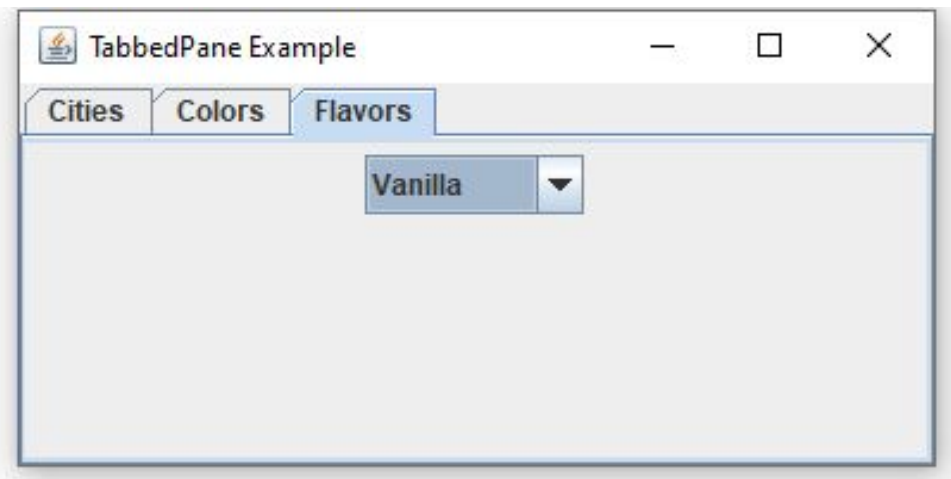
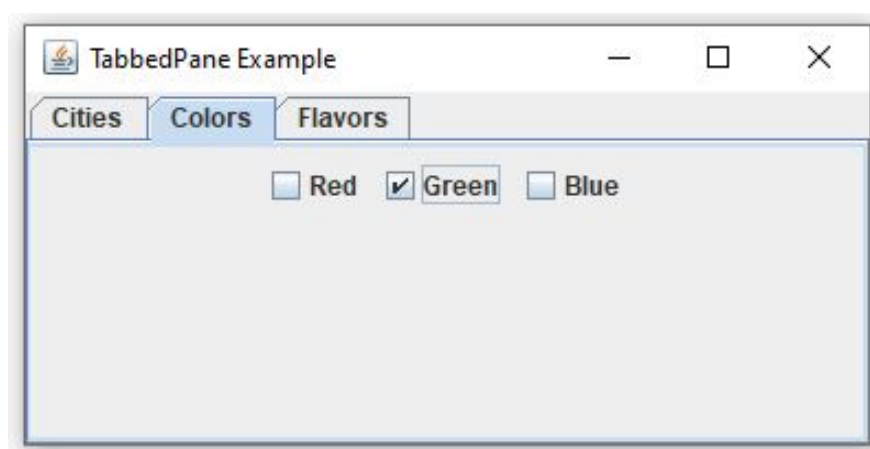
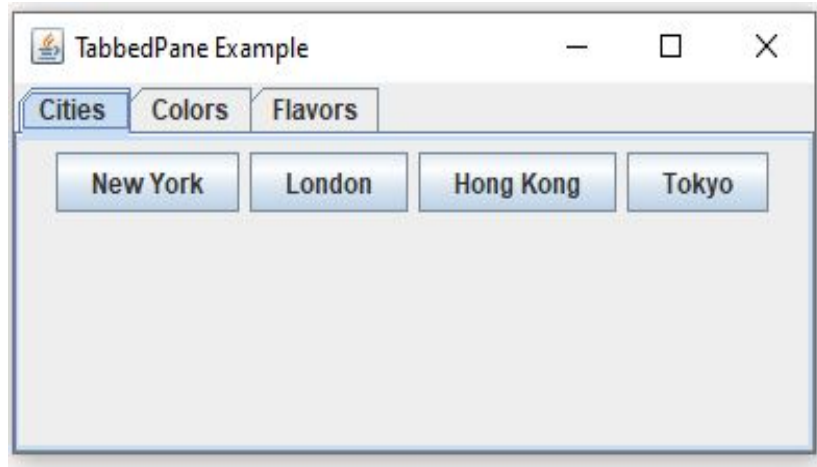
```
public static void main(String[] args) {  
    new JTabbedPaneDemo();  
}  
  
}  
  
class CitiesPanel extends JPanel {                                // Panel for "Cities" tab  
    public CitiesPanel() {  
        add(new JButton("New York"));  
        add(new JButton("London"));  
        add(new JButton("Hong Kong"));  
        add(new JButton("Tokyo"));  
    }  
}
```

// Panel for "Colors" tab

```
class ColorsPanel extends JPanel {  
    public ColorsPanel() {  
        add(new JCheckBox("Red"));  
        add(new JCheckBox("Green"));  
        add(new JCheckBox("Blue"));  
    }  
}
```

// Panel for "Flavors" tab

```
class FlavorsPanel extends JPanel {  
  
    public FlavorsPanel() {  
  
        JComboBox<String> jcb = new JComboBox<>();  
  
        jcb.addItem("Vanilla");  
  
        jcb.addItem("Chocolate");  
  
        jcb.addItem("Strawberry");  
  
        add(jcb);  
  
    }  
  
}
```



JScrollPane

JScrollPane is a **Swing container** that provides a scrollable view of another component. It automatically shows **vertical and/or horizontal scrollbars** when the content is too big to fit in the visible area (called the **viewport**).

- A JPanel containing **many components** (like 400 buttons).
- This panel is **larger than the visible window**.

You wrap this panel inside a JScrollPane, and when the panel overflows, scrollbars appear **automatically**, allowing the user to scroll through the content.



```
import java.awt.*;

import java.awt.event.*;

import javax.swing.*;

public class JScrollPaneExample implements ActionListener {

    JFrame frame;

    JPanel panel;

    JScrollPane scrollPane;

    JButton dummyButton;

    public JScrollPaneExample() {

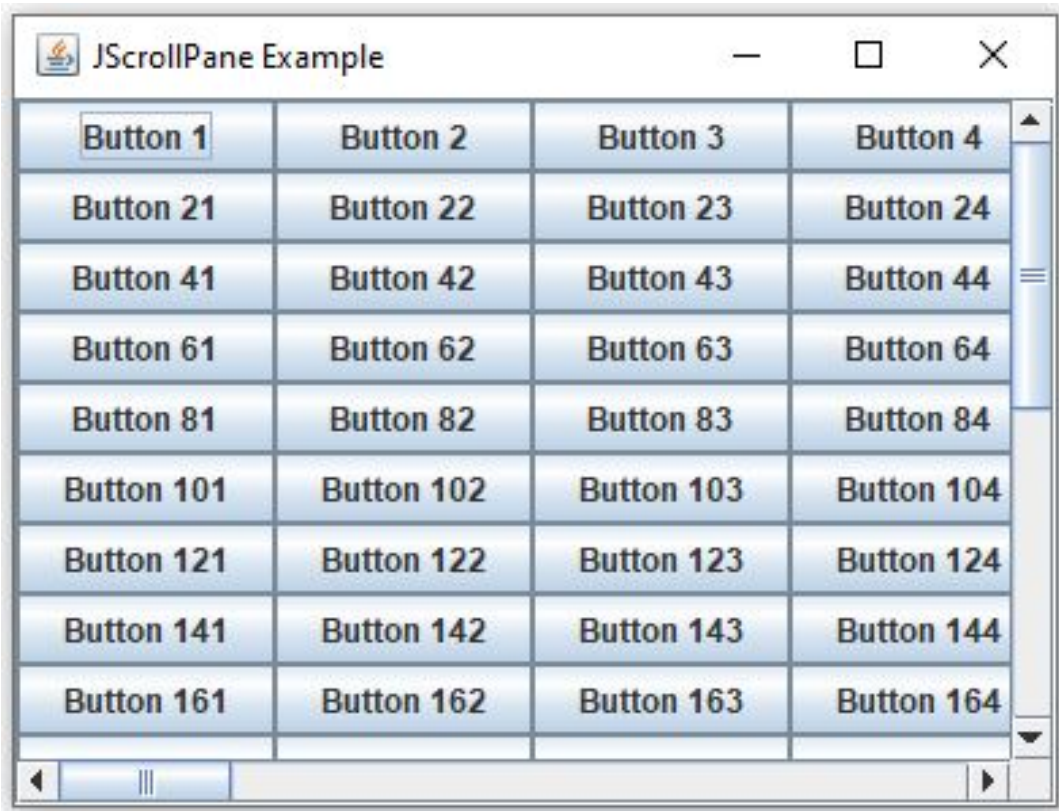
        frame = new JFrame("JScrollPane Example");

        frame.setSize(400, 300);

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
panel = new JPanel();  
panel.setLayout(new GridLayout(20, 20)); // 20x20 = 400 buttons  
int count = 1;  
for (int i = 0; i < 400; i++) {  
    panel.add(new JButton("Button " + count++));  
}  
scrollPane = new JScrollPane(panel);  
frame.add(scrollPane);  
frame.setVisible(true);  
}
```

```
public void actionPerformed(ActionEvent e) {  
    // No action needed for this example  
}  
  
public static void main(String[] args) {  
    JScrollPaneExample d= new JScrollPaneExample();  
}  
}
```



JList

- JList is a Swing component used to **display a list of items**, where the user can **select one or more items**.
- It is a part of the javax.swing package.

Common Use Cases:

- Selection menus (e.g., list of cities, names, files).
- Input filtering or category selection in UI.
- Multi-item selection in user interfaces.

Constructor:

JList(Object[] items)

Creates a list with the items provided in the array.

Common Methods:

Method	Description
<u>setSelectionMode()</u>	Set how many items can be selected (e.g., SINGLE, MULTIPLE).
<u>addListSelectionListener()</u>	Adds an event listener for selection changes.
<u>getSelectedIndex()</u>	Returns the index of the selected item.
<u>getSelectedValue()</u>	Returns the actual value (object) selected.

Wrapping in JScrollPane:

Since the list can be long, it's often wrapped in a JScrollPane so the user can scroll through the list:

```
JScrollPane scrollPane = new JScrollPane(jlst);
```

Event Handling:

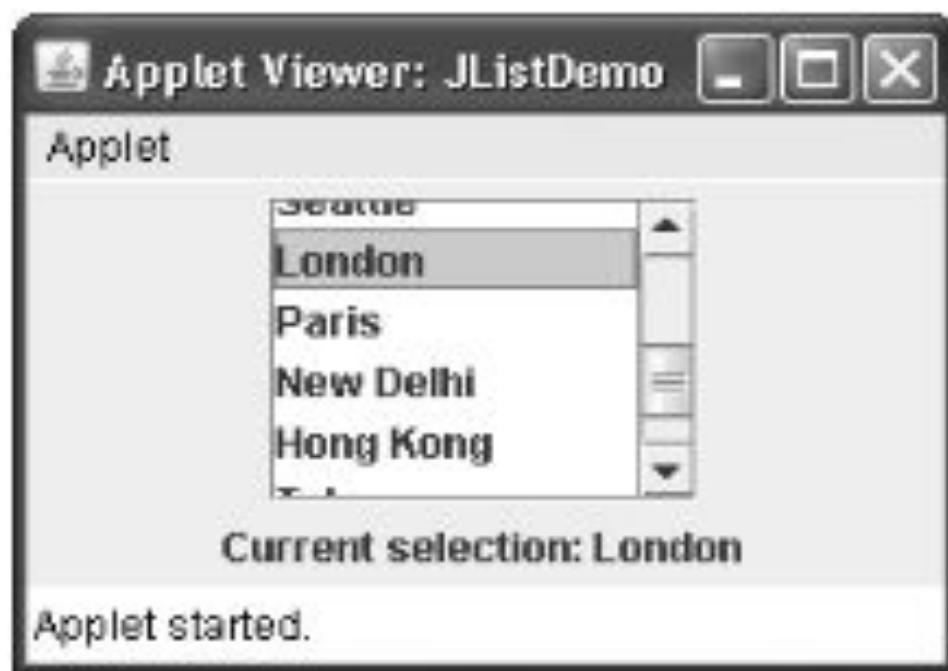
- JList generates a **ListSelectionEvent** when a user selects or changes an item.
- You must implement **ListSelectionListener** and override the method:

public void valueChanged(ListSelectionEvent e)

Selection Modes:

Set by `setSelectionMode()` using:

- **ListSelectionModel.SINGLE_SELECTION**
- **ListSelectionModel.SINGLE_INTERVAL_SELECTION**
- **ListSelectionModel.MULTIPLE_INTERVAL_SELECTION**



```
import javax.swing.*;

import javax.swing.event.*;

import java.awt.*;

public class JListDemo implements ListSelectionListener {

    JList<String> jlst;

    JLabel jlab;

    JScrollPane jscrlp;

    JFrame frame;

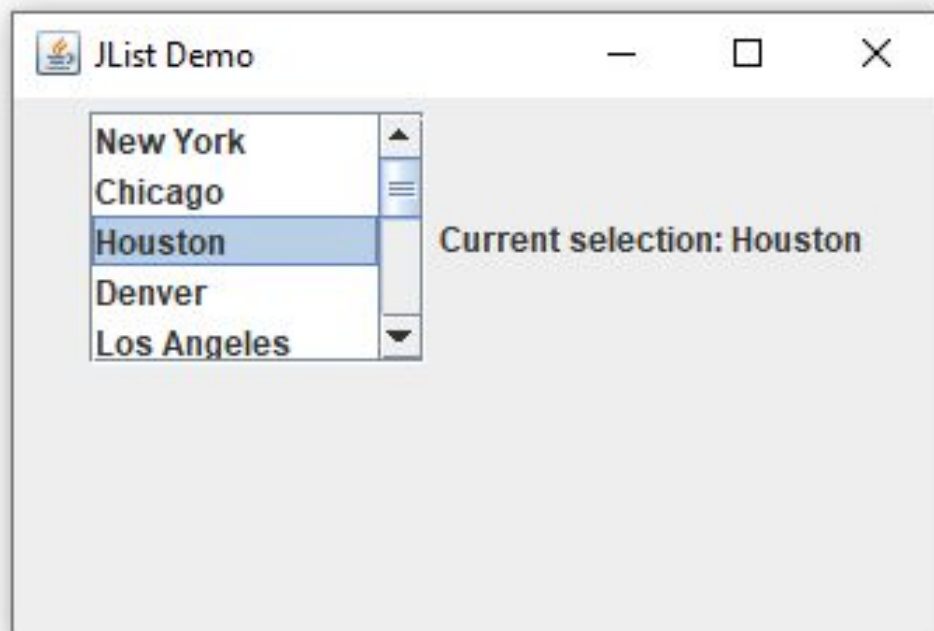
    String[ ] cities = {

        "New York", "Chicago", "Houston", "Denver", "Los Angeles", "Seattle", "London",
        "Paris", "New Delhi", "Hong Kong", "Tokyo", "Sydney" };
}
```

```
public JListDemo() {  
  
    frame = new JFrame("JList Demo");  
  
    frame.setSize(250, 200);  
  
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
  
    frame.setLayout(new FlowLayout());  
  
    // Create JList  
  
    jlst = new JList<>(cities);  
  
    jlst.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);  
}
```

```
jscrollp = new JScrollPane(jlst);  
  
jscrollp.setPreferredSize(new Dimension(120, 90));  
  
jlab = new JLabel("Choose a City");  
  
jlst.addListSelectionListener(this);  
  
frame.add(jscrollp);  
  
frame.add(jlab);  
  
frame.setLocationRelativeTo(null); // Center the frame  
  
frame.setVisible(true);  
  
}
```

```
public void valueChanged(ListSelectionEvent e) {  
    int idx = jlst.getSelectedIndex();  
    if (idx != -1)  
        jlab.setText("Current selection: " + cities[idx]);  
    else  
        jlab.setText("Choose a City");  
}  
  
public static void main(String[] args) {  
    JListDemo d= new JListDemo();  
}  
}
```



JComboBox

- A **combo box** is a GUI component that combines a **drop-down list** with a text field.
- It shows **one selected item** at a time.
- When clicked, it drops down a list of available options to choose from.
- Users can select an item from the list.
- It can optionally allow the user to type in the text field (editable combo box).

You create a combo box with an array of items:

```
JComboBox combo = new JComboBox(Object[] items);
```

You can dynamically add items via `addItem(Object obj)` for mutable combo boxes.

When a user selects an item:

- An **ActionEvent** is generated.
- Also, **ItemEvents** occur for deselection and selection of items.

You get the selected item by:

```
Object selected = combo.getSelectedItem();
```

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;

class JComboBoxDemoExample implements ActionListener {

    static JFrame frame;

    JComboBox<String> comboBox;

    JLabel label;

    Public JComboBoxDemoExample( ){

        frame = new JFrame("JComboBox Demo");

        frame.setSize(300, 150);

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
frame.setLayout(new FlowLayout());
```

```
String[ ] flags = { "France", "Germany", "Italy", "Japan" };
```

```
JComboBox<String> comboBox = new JComboBox<>(flags);  // Create JComboBox  
with countries
```

```
// Default image label
```

```
label = new JLabel(new ImageIcon("France.gif"));
```

```
// Add components
```

```
frame.add(comboBox);
```

```
frame.add(label);
```

```
comboBox.addActionListener(this);
```

```
frame.setVisible(true);  }
```

```
public void actionPerformed(ActionEvent e) {  
  
    // Get selected country and update image  
  
    String selectedCountry = (String) comboBox.getSelectedItemAt()  
    label.setIcon(new ImageIcon(selectedCountry + ".gif"));  
  
}  
  
}  
  
class Demo  
  
{  
  
public static void main(String[] args) {  
  
    JComboBoxDemoExample e=new JComboBoxDemoExample();  
  
} }  
}
```



